

EVALUACION	PRACTICA	
MATERIA	DISEÑO DE APLICACIONES CON DESARROLLO GENERATIVO	
CARRERA	ANALISTA EN TECNOLOGIAS DE INFORMACIÓN / ANALISTA PROGRAMADOR	

Ejercicio 1

Cadena de cines y funciones

Una cadena opera varios cines; cada cine tiene múltiples salas con butacas numeradas. Las películas se programan en funciones (día/hora/sala) con venta anticipada de butacas. Los espectadores están registrados en el sistema y sacan entradas para una función, en una sala y una butaca. Los consumos del espectador se asocian a la entrada.

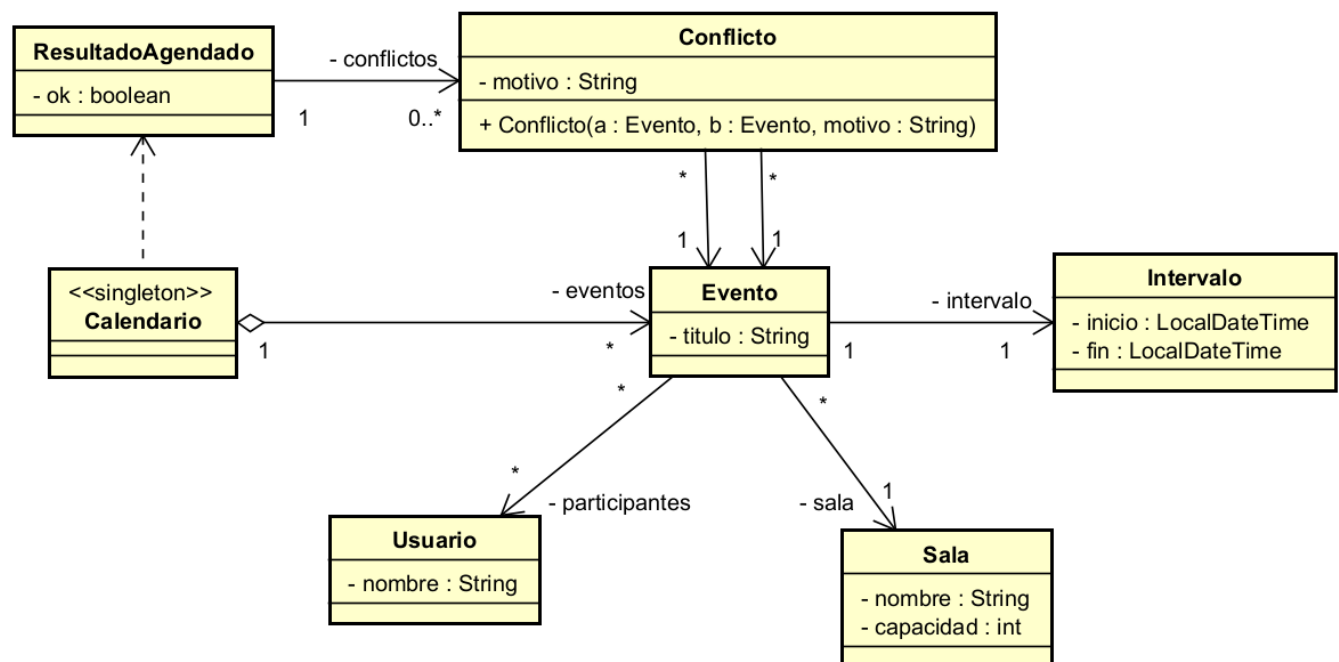
Se pide:

- Elaborar un diagrama conceptual de dominio, incluyendo entidades, sus relaciones y multiplicidad.

Ejercicio 2

Conflictos de agenda en un calendario

El siguiente diagrama de clases corresponde al modelo actual y la funcionalidad que ya se encuentra implementada:



Nota: todas las clases cuentan con getters para todos sus atributos. También cada clase cuenta con un `equals(Object o) : boolean` ya implementado.

Implementar el siguiente requerimiento

Requerimiento: agendar un nuevo evento

Dado un nuevo evento definido con un intervalo, una sala y una lista de participantes, agendar el evento en el calendario.

Como resultado de esta operación se espera un objeto `ResultadoAgendado`, que contiene propiedad `ok` que indica si fue agendado o no y opcionalmente una lista de conflictos.

Condiciones:

- El evento es agregado a la agenda salvo que ocurra alguna de las siguientes situaciones:
 - o El título es nulo o vacío;
 - o El fin es anterior a inicio;
- El sistema debe generar un `Conflicto` para cada una de estas situaciones:
 - o Si la sala admite menos capacidad que la cantidad de participantes. Motivo: "Capacidad de sala insuficiente".
 - o Si el evento se solapa con el intervalo de un evento existente y comparten la misma sala. Motivo: "Solapamiento en sala [nombre de la sala]"
 - o Si el evento se solapa con el intervalo de un evento existente y comparten alguno de los participantes. Motivo: "Participante duplicado en horarios solapados".

El siguiente es el endpoint definido en el presentador:

```
@RestController
@RequestMapping("/evento")
public class CalendarioPresentador {

    @PostMapping("/agendar")
    public Collection<Comando> agendar(@RequestParam String titulo,
                                      @RequestParam LocalDateTime inicio,
                                      @RequestParam LocalDateTime fin,
                                      @RequestParam int indexSala,
                                      @RequestParam List<int> idxParticipantes) {

        Evento e = new Evento(titulo);
        e.setIntervalo(new Intervalo(inicio, fin));
        e.setSala(Sistema.getInstancia().getSalas().get(indexSala));
        e.setPartipantes(Sistema.getInstancia().getParticipantesPorIdx(idxParticipantes));

        // implementar requerimiento aquí

    }
}
```

Se pide:

- Implementar en java el caso de uso descripto.

Consideraciones:

- Tener en cuenta que todos los métodos que aparecen en el diagrama de clases ya se encuentran implementados y algunos podrían ser útiles para su solución. **Utilizar**

estrictamente el diagrama de clases dado en la letra como base para su solución. Se podrán agregar nuevas clases, métodos y relaciones.

- No es necesario implementar getters/setters.
- No puede haber lógica/código duplicado.

Nota:

- LocalDateTime tiene un método `isBefore (LocalDateTime otro)` que permite compararse con `otro` objeto

Evaluación:

- Ejercicio 1: 8 puntos
- Ejercicio 2: 12 puntos